



A self-supervised BEiT model with a novel hierarchical patchReducer for efficient facial deepfake detection

Aneesa Al Redhaei¹ · Salam Fraihat¹ · Mohammed Azmi Al-Betar^{1,2}

Accepted: 30 May 2025 / Published online: 12 June 2025
© The Author(s) 2025

Abstract

The spread of deepfake technology has become a growing concern, especially with the rapid advancement of generative models. This progress has made it increasingly difficult to distinguish between real and fake facial videos. This poses serious threats to security, privacy, and the spread of misinformation. Despite the existence of deepfake detection models, many suffer from high computational costs. This makes them impractical for deployment in resource-constrained environments. To address this challenge, this paper proposes a solution that accurately identifies deepfakes while reducing computational complexity. To achieve this, a deepfake detection system using an improved version of a Self-Supervised BEiT, called BEiT-HPR (Hierarchical PatchReducer), is proposed. The enhancement adds a Hierarchical PatchReducer layer to reduce the number of patches in successive encoder blocks. This reduces computational complexity while maintaining high detection accuracy. Additionally, training speed increases by over 50%, while the number of parameters is reduced by 63.4%. The BEiT-HPR model was tested using three publicly available benchmark datasets: FaceForensics++ (FF++), Celeb-DF, and the Deepfake Detection Dataset (DFD). The evaluation results revealed that reducing the model complexity by 62.2% allowed the proposed model to achieve an accuracy of 83.92% on FF++, 97.59% on Celeb-DF, and 98.25% on DFD. These findings emphasize the importance of computationally efficient deepfake detection methods that maintain high detection accuracy while reducing the burden of heavy computation. Therefore, it offers a scalable solution for identifying deepfakes across diverse datasets.

Keywords Facial deepfake · Vision transformer · Self-supervised learning · PatchReducer layer · Computational complexity

1 Introduction

The rapid and exponential advancement of recent technologies in artificial intelligence (AI) and deep learning (DL) has led to significant advances in media content creation. This includes the development of hyper-realistic artificial media known as “deepfakes”. Deep-

fakes can modify facial features, change speech, or even create entirely false personalities. This makes it more challenging to distinguish between authentic and synthetic content. The phenomenon gained widespread attention in 2017 with the release of the first deepfakes, which used AI to replace the faces of celebrities with inappropriate video content (Deshmukh and Wankhade 2020). This marked the beginning of a rise in concerns regarding the potential misuse of the technology.

According to the UAE Ministry of AI (AIGov 2021), deepfake technology is becoming more powerful and accessible, making it easier to simulate very realistic fake visual content, including videos and images. These manipulations can harm individuals' reputations and disrupt organizations and national stability by fabricating actions, events, or behaviors. Deepfakes can falsely show people involved in activities that never took place. On a local level, they can influence public opinion, manipulate communities, and create public agendas. If not quickly identified, deepfakes can also damage international relations and pose reputational threats to countries.

According to Reuters news (Ulmer and Tong 2023), the number of deepfakes has grown rapidly, doubling every six months. In 2023, around 500,000 deepfake videos and audio clips were shared globally on social media. By 2025, this number is expected to reach 8 million, following the same pattern of doubling every six months (Ulmer and Tong 2023). As a result, building reliable systems to identify deepfakes is essential to reducing the risks they pose, including threats to privacy, security, and societal trust.

In response to these challenges, deepfake detection systems have emerged as crucial tools with diverse applications across various sectors (The IoT 2024). In media and journalism, these systems help verify the authenticity of news footage, combating misinformation and maintaining journalistic integrity. Law enforcement agencies utilize deepfake detection to verify video evidence for criminal investigations, ensuring the reliability of digital evidence in legal proceedings. Online platforms employ these technologies for content moderation, monitoring and removing deepfake content to maintain the credibility of user-generated content. In the political sphere, detection systems protect against misleading deepfake campaigns, safeguarding democratic processes. Additionally, organizations use deepfake detection as part of their cybersecurity measures to guard against identity theft and fraud attempts using manipulated media.

In recent years, to address the challenges of detecting deepfakes, several deep learning techniques have been developed. These methods including advanced approaches like long short-term memory (LSTM) networks, recurrent neural networks (RNNs), and hybrid models have been used to identify alterations in images and videos (Almars 2021). Among these methods, Vision Transformer-based (ViT) (Dosovitskiy et al. 2020) methods are the most used (Wang et al. 2024). ViTs are a type of deep learning model that performs well in image recognition and classification tasks. ViTs have demonstrated notable success across various domains such as healthcare, where they have been applied to medical image processing for tasks like skin cancer detection (Pacal et al. 2025; Ozdemir and Pacal 2025) and detection of brain stroke (Bayram et al. 2025), and agriculture, where they are used for crop disease detection and monitoring (Pacal 2024; Pacal et al. 2024). Their ability to capture long-range dependencies in images makes them particularly effective in tasks that require a global understanding of the image content. Moreover, they can capture important patterns in images, which makes them useful for detecting deepfakes.

However, despite their effectiveness, these methods require large amounts of labeled data for training, which is often impractical in deepfake detection (Nguyen et al. 2024; Gerritsen 2022; Dagar and Vishwakarma 2024). Generating labeled data for a new learning task is one of the primary challenges in applying advanced deep learning methods (Li et al. 2024). For deepfake detection, collecting sufficient labeled data is both costly and time-consuming. This limitation makes ViTs and their improved versions less attractive compared to other methods that can perform well with smaller datasets.

Additionally, ViT models rely on the self-attention mechanism, which has quadratic complexity relative to the input size. This makes ViTs computationally intensive, especially when processing high-resolution images (Qin et al. 2023). As a result, many deepfake detection studies avoid using vision transformer-based methods or emphasize their high computational cost (Xu et al. 2023; Essa 2024; Mauricio et al. 2023). This limitation affects the scalability and suitability of ViTs for large-scale training scenarios, making other deep learning methods more favorable.

These challenges show the need for new approaches to make ViTs and their improved versions more practical for deepfake detection. Reducing their computational cost and data requirements, while keeping their performance high. This would make them more useful in real-world situations.

To address the challenges of deepfake detection, this paper proposes a self-supervised deepfake detection pipeline that integrates several existing techniques in a novel way. The key contributions of this research work are as follows:

- Improve deepfake detection accuracy with efficient processing, by the combination of techniques such as weight initialization from Self-Supervised pre-trained ImageNet models, keyframe extraction, frame balancing, and minimal preprocessing.
- Introduce an Enhanced Vision Transformer (ViT) model, called Bidirectional Encoder representation from Image Transformers (BEiT) (Bao et al. 2021). The model incorporates a PatchReducer layer, an advanced version of the PatchMerger layer (Renggli et al. 2022), strategically placed within the encoder. This optimization reduces computational complexity while maintaining high detection performance.
- Evaluate the proposed model using a cross-data technique, where it is trained on one dataset and tested on different datasets. This approach ensures robust generalization by preventing overfitting to specific dataset characteristics and validating the model's effectiveness across diverse deepfake sources.
- A detailed review of existing deepfake detection methods is provided, with an analysis of the gaps in current approaches.

The motivation behind this research is to develop a deepfake detection system that is not only accurate but also efficient, making it suitable for real-world applications where computational resources may be limited. By combining these techniques in a novel way, this work provides a scalable solution to deepfake detection with minimal data and efficient processing.

The remainder of this paper is structured as follows: Section 2 summarizes the recent studies in the deepfake detection domain. The methodology is presented in Section 3. Section 4 presents and discusses the results. The last section states the conclusion and the future directions.

2 Literature review

As deepfakes became more popular on social media in 2017, concerns about their potential risks grew. Consequently, this led to an increase in demand for appropriate deepfake detection methods in early 2018. One of the initial techniques developed back then was based on distinguishing how frequently a person blinks on deepfake videos compared to real ones (Li et al. 2018). Additionally, some early detection methods used statistical tools like photoreponse nonuniformity (PRNU) to spot fake videos. In the work by (Koopman et al. 2018), PRNU analysis was employed to identify a clear statistical difference in the average normalized cross-correlation scores between real and fake videos. However, as generative adversarial networks (GANs) models continue to evolve, deepfakes are becoming increasingly realistic. Therefore, more sophisticated detection methods are necessary. Despite some machine learning techniques for detecting deepfakes that have been developed, deep learning methods are currently the most widely used in the field of deepfake detection (Rana et al. 2022; Malik et al. 2022). The deep learning methods include simple traditional deep neural networks and more advanced deep learning methods, including CNN-based detection methods, Transformer-based detection methods, and Hybrid-based detection methods.

2.1 Deep neural network-based detection methods

Deep Neural Networks (DNNs) are powerful tools for deepfake detection, as they can analyze complex patterns in images and videos to identify manipulated content. Various studies have demonstrated the effectiveness of DNN-based approaches in this field. For instance, Lee and Kim (2021) used a six-layer DNN to analyze changes in visual features, such as mean squared error and luminance across video frames, achieving accuracies of 97.39% on Face2Face, 95.65% on FaceSwap, and 96.55% on DFDC. Similarly, Sahla Habeeba et al. (2021) focused on detecting artifacts in the eyes and teeth regions using a three-layer DNN, reporting 92% accuracy on UADFV and 86% on DFD datasets. In another study, Demir and Ciftci (2021) utilized eye and gaze features analyzed in spatial, temporal, and spectral domains, with a four-layer DNN achieving 92.48% on FaceForensics++, 88.35% on Celeb-DF, and 99.27% on DeeperForensics. Additionally, Saravana Ram et al. (2023) proposed an enhanced Deep Belief Network (DBN) combined with fuzzy clustering for feature extraction, achieving over 96% accuracy across datasets like Face2Face and 100KFaces.

2.2 Convolutional neural network-based detection methods

Convolutional Neural Networks (CNNs) have revolutionized image processing by effectively learning and extracting features from visual data. Their hierarchical structure enables them to capture complex visual representations, making them highly suitable for deepfake detection. Early studies employed pre-trained models like VGG, ResNet, Inception, MobileNet, DenseNet, and Xception, fine-tuned on deepfake datasets to detect artifacts and inconsistencies introduced during the deepfake generation process (Li and Lyu 2018; Amerini et al. 2019; Suratkar et al. 2020; Zhang and Liu 2020). More advanced CNN architectures later incorporated attention mechanisms and ensemble learning techniques to further enhance detection accuracy. One notable CNN-based model is MesoInception (Afchar et al. 2018), inspired by InceptionNet, which captures mesoscopic image features. It achieved over 98%

and 95% detection accuracy on FaceForensics++ Deepfake and Face2Face datasets, respectively. Further improvements were made by Mathews et al. (2023) with a framework using MesoInception-Net on the DFIM-HQ dataset, achieving 99.87% accuracy. Similarly, SiamNet (Kingra et al. 2023) used a Siamese-like network to analyze camera noise patterns, achieving accuracy rates exceeding 98% on multiple datasets, including FaceForensics++ and Celeb-DF. Additionally, Patel et al. (2023) proposed a Dense CNN (D-CNN) for binary classification, achieving over 95% accuracy across FFHQ and Celeb-A datasets. Abdullah and Ali (2023) enhanced detection by integrating texture and color features, achieving 99.8% accuracy on Celeb-A. Using the same data, Chougule et al. (2024) proposed a new GAN-based method with a U-Net generator and a DCGAN discriminator to improve low-resolution face recognition. The model was trained for 4x super-resolution to increase image clarity while preserving facial identity for efficient deepfake detection. Wang et al. (2023) introduced Tception, a two-stream Xception-based network combining RGB and noise-space features, achieving 97.47% on Celeb-DF. Omar et al. (2024) developed an ensemble CoAtNet model integrating convolutional and self-attention mechanisms, achieving over 97% accuracy on the Celeb-DF dataset. Lu et al. (2023) incorporated long-distance attention to highlight key facial regions, achieving 99.51% on FaceForensics++. Furthermore, Zhao et al. (2021) introduced a multi-attentional deepfake detection network that improves accuracy by using multiple spatial attention heads to detect artifacts. It enhances textural features in shallow layers and combines low-level and high-level features with attention maps. The method achieved top performance on benchmark datasets, including FaceForensics++, Celeb-DF, and DFDC. Qin et al. (2023) proposed a multi-branch Xception-based framework for fine-grained feature analysis, recording over 98% accuracy on FaceForensics++ and Celeb-DF. Fang et al. (2023) introduced a temporal framework combining a 3D attention mechanism and time-sequence graph convolutional network, achieving notable generalization capabilities with 84.03% and 72.61% accuracy on the Celeb-DF and DFDC datasets. Lastly, Guan et al. (2023) proposed a meta-learning framework with EfficientNet-b4 for few-shot scenarios, achieving strong cross-data generalization, with 81.70% accuracy on Google DF and 64.40% on DFDC.

2.3 Transformer-based detection methods

Transformers have significantly advanced multiple technological domains, starting with their introduction in natural language processing (NLP) by Vaswani et al. (2017). Expanding into computer vision, Vision Transformers (ViTs) treat images as sequences of patches, leveraging self-attention to understand spatial relationships (Dosovitskiy et al. 2020). This approach has proven highly effective for deepfake detection, often surpassing traditional CNNs on extensive datasets (Arshed et al. 2023). Stand-alone ViT models have shown exceptional results. For instance, Arshed et al. (2023) fine-tuned a pre-trained ViT on FFHQ and StyleGAN-generated deepfakes, achieving 100% accuracy. Similarly, Ghita et al. (2024) customized ViTs for the OpenForensics dataset, attaining 90% accuracy. Thing (2023) reported varying accuracies on datasets such as FaceForensics++ (75.73%), Google DFD (87.20%), and DeeperForensics (99.73%). Enhanced ViT architectures like the Swin Transformer (Liu et al. 2021) have further improved detection. For example, Mishra et al. (2024) achieved 97.91% accuracy on Celeb-DF, and Thing (2023) explored both Swin Transformer and BEiT frameworks, yielding similar success. Computationally efficient

adaptations presented in shallow ViT by Usmani et al. (2024) significantly reduced parameters and achieved 88.52% accuracy on the RFFD dataset.

Advanced approaches have incorporated spatial and temporal features. Ramadhani et al. (2024) proposed a system using video vision transformers with attention modules, achieving over 90% accuracy on FaceForensics++. Khormali and Yuan (2022) introduced re-attention mechanisms to enhance diversity in attention maps, attaining 98.18% accuracy on FaceForensics++. Methods like distillation-based learning (Zhu et al. 2024) have addressed challenges in detecting compressed deepfakes, with accuracies exceeding 95% on FaceForensics++. Self-supervised learning (SSL) has emerged as a key trend. Khan and Dang-Nguyen (2023) employed DINO, a transformer-based SSL model, demonstrating strong inter-data evaluation results. Masked image modeling (MIM) techniques like Self-Supervised Auxiliary Tasks (SSAT) (Das et al. 2024) significantly enhanced ViT performance, achieving over 96% accuracy on DFDC datasets. Similarly, Chen et al. (2022) introduced SSL with Swin Transformers, showing robust adaptability and an AUC exceeding 80% across benchmark datasets.

2.4 Hybrid-based detection methods

Hybrid-based detection methods combine multiple computational models, enhancing deepfake detection by integrating neural networks, convolutional layers, and transformer architectures. For example, (Mirzayev and DİBEKLİOĞLU 2024) proposed combining CNNs with LSTMs, using motion magnification-inspired feature manipulation to enhance frame representations, achieving 86% accuracy on Celeb-DF. (Heidari et al. 2024) introduced a federated learning framework using SegCaps, CNNs, and capsule networks, significantly improving performance and achieving accuracy rates above 97% on datasets like CelebDF and FaceForensics++. (Tolosana et al. 2022) applied fusion techniques at both system and facial region levels, achieving an AUC score above 99% across three datasets. (Lin et al. 2023) integrated multi-scale convolutions and a vision transformer, reaching 99.8% accuracy on Celeb-DF and 88.47% on DFDC. Similarly, (Wang et al. 2023) combined convolutional pooling and re-attention mechanisms, yielding 92.11% accuracy on FaceForensics++. (Khormali and Yuan 2024) employed a Vision Transformer-based feature extractor and a Graph Convolutional Network, achieving 99.47% accuracy on Celeb-DF. Finally, (Kavthekar et al. 2023) introduced a hybrid model using Masked Autoencoder ConvNeXt and Multiscale Vision Transformer, achieving accuracy ranging from 80.22% to 98.1%. These hybrid approaches offer effective solutions by leveraging the strengths of multiple models.

2.5 Literature analysis and limitations

The literature review provides a detailed exploration of various methods for video deepfake detection, from early deep learning models to more advanced approaches such as CNNs, ViTs, and hybrid architectures. These methods have progressively improved in addressing the growing sophistication of manipulated content, with CNNs and ViTs being among the most commonly utilized techniques. Table 1 summarizes the limitations of the existing methods and how the proposed BEiT-HPR model will address them.

Table 1 Comparison of Detection Methods and Their Limitations Addressed by BEiT-HPR

Detection method	Limitations	Limitations addressed by BEiT-HPR
DNN-Based	Simple architectures, require extra preprocessing steps to avoid overfitting, especially with small datasets	BEiT-HPR integrates pre-trained models and frame extraction techniques, reducing the need for extensive preprocessing and overfitting, and efficiently detecting deepfakes with minimal labelled data
CNN-Based	Struggles to understand relationships between distant pixels and may face challenges in localizing key regions in images	The hierarchical PatchReducer layer in BEiT-HPR enables the model to better capture global and local features, improving accuracy without relying solely on attention mechanisms
Transformer-Based	High computational demands, especially for large-scale training scenarios and cross-data evaluations. Also, requires large data for training	BEiT-HPR reduces computational complexity through hierarchical patch reduction and self-supervised learning, enabling computationally efficient large-scale training scenarios even on devices with limited resources
Hybrid-Based	Require complex training pipelines and significant computational resources	BEiT-HPR combines the strengths of ViTs with the PatchReducer layer, providing a simpler yet effective solution that reduces computational costs while maintaining accuracy

In summary, while ViTs and their enhanced versions have demonstrated effectiveness, their computational demands often limit their practical use, especially in large-scale training scenarios or on devices with limited processing power. Implementing techniques such as transfer learning is essential, yet the reviewed works show a notable gap in the application of transformer-based transfer learning methods. Although a few studies have explored self-supervised learning models as a form of transfer learning (Das et al. 2024; Chen et al. 2022; Khormali and Yuan 2024; Khan and Dang-Nguyen 2023; Kavthekar et al. 2023; Thing 2023), performance often depends on significant computational resources, limiting their broader applicability. Also, one study done by Usmani et al. (2024) showed its aim to reduce the complicity of the standard ViT architecture by changing the image shape, batch size, number of attention heads, and linear projection dimensions. However, reducing these parameters can lower the computational cost, but it may also make the model less able to capture important details. Also, it may reduce its ability to generalize, especially in more complex or diverse situations.

Based on the limitations discussed in the presented literature, which are also mentioned in a recent survey paper (Mubarak et al. 2023), the challenge of balancing detection accu-

racy with computational efficiency persists. This highlights the need for solutions that can effectively balance model performance and computational efficiency. This research work seeks to address these challenges by developing a modified version of the BEiT model (Bao et al. 2021), aiming to reduce computational complexity while maintaining high detection accuracy. This contribution is expected to enhance the practical utility of self-supervised learning in detecting manipulated faces, particularly in environments with constrained resources, providing a more efficient tool to counter the growing threat of deepfakes.

3 Methodology

The proposed system leverages state-of-the-art ViT technology, enhanced through self-supervised learning methods. Central to this system is the utilization of a pre-trained BEiT model, which is further refined to enhance its deepfake detection capabilities. As illustrated in Fig. 1, the system operates in three primary stages: data preparation, model training, and performance evaluation.

3.1 Datasets

The proposed deepfake detection system is trained on three well-known benchmark datasets: FaceForensics++ (FF++) (Rossler et al. 2019; Li et al. 2019), Celeb-DF (Li et al. 2020), and the DeepFake Detection Dataset (DFD) (Dufour and Gully 2019). These datasets are selected for their diversity in deepfake generation techniques, video resolutions, and real-world scenarios, which contribute to the robustness and generalizability of the model.

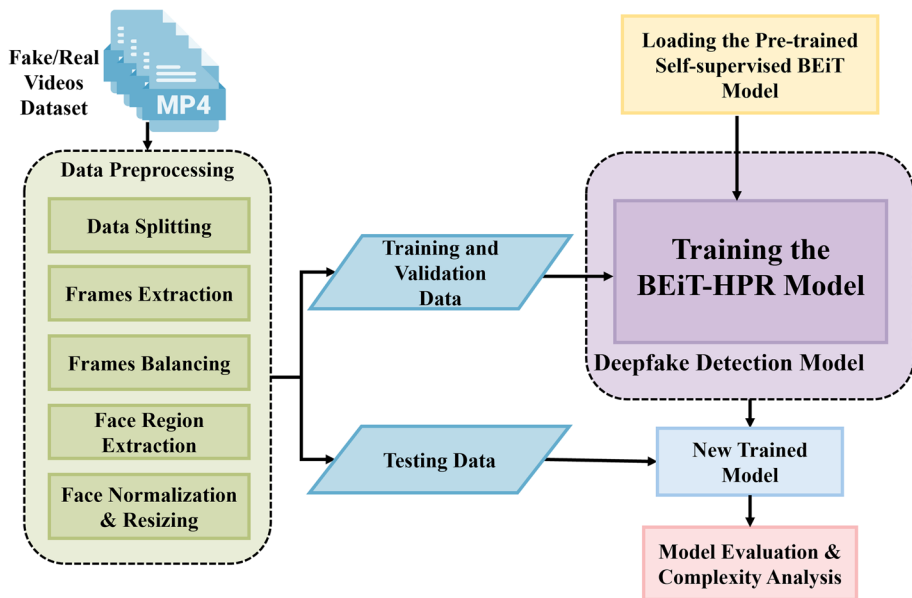


Fig. 1 The proposed system architecture

FF++ contains 5,000 fake videos and 1,000 real videos, with the real videos sourced from YouTube. The fake videos are generated using various facial manipulation techniques, including DeepFakes (DF), Face2Face (F2F), FaceSwap (FS), NeuralTextures (NT), and FaceShifter (FSh), which encompass both deep learning-based and computer graphics-based methods. The dataset also includes three types of video quality: Raw (no compression), High Quality (HQ) with low compression, and Low Quality (LQ) with high compression. For the current study, HQ videos are used to balance memory usage and video quality. The video resolutions in FF++ range from 480p to 1080p, with 480p being the most common. The diversity of manipulation techniques and video resolutions in FF++ makes it a valuable benchmark for deepfake detection research (Rossler et al. 2019; Li et al. 2019).

Celeb-DF consists of 5,639 deepfake videos generated from 590 original video sequences of 59 celebrities, with each video typically lasting around 13 s at 30 frames per second. The real videos are sourced from YouTube interviews and reflect a variety of genders, ages, and ethnicities. The deepfake videos are generated using an advanced synthesis method that swaps faces among the 59 individuals. To improve the visual fidelity of the manipulated faces, the dataset creators enhance the resolution of the deepfake faces to 256×256 pixels using advanced encoder-decoder models. This high-resolution manipulation increases the difficulty of distinguishing fake videos from real ones, making Celeb-DF a challenging dataset for deepfake detection (Li et al. 2020).

The DeepFake Detection Dataset (DFD) was created by Google and later merged into the FF++ dataset. It includes 360 real and 3,068 deepfake videos, featuring a wide range of ethnicities, lighting conditions, and scene complexities. The dataset was generated using undisclosed deepfake technologies, which adds to its complexity and makes it less commonly used in prior research. The variety in the DFD dataset further contributes to the model's ability to generalize across different types of deepfake manipulations and environmental conditions (Dufour and Gully 2019).

3.2 Data preprocessing

This section explains the preprocessing steps: data splitting, frame extraction, frame balancing, face detection, and face normalization with resizing.

3.2.1 Data splitting

The dataset is divided into three subsets: training, validation, and testing. The training set is used to train the model. During the training phase, the validation set helps tune hyperparameters and prevent overfitting. The testing set is used for final evaluation.

Each of the three datasets is split into non-overlapping training, validation, and test sets. For the FF++ dataset, the provided splits are used, while for the other datasets, the videos are divided to allocate approximately 70% for training, 10% for validation, and 20% for testing. This non-overlapping division is essential for ensuring that the model's performance evaluation is unbiased and reflects its ability to generalize to unseen data. Generalizability, in this context, refers to the model's ability to accurately detect deepfakes on videos or frames that it has never encountered during training. By isolating the testing set, the model's performance is assessed under conditions that mimic real-world scenarios, where the deepfake detection system must handle entirely new and unseen content. Furthermore, the use of a

Table 2 Video splits

Dataset	Set	Fake videos	Real videos	Fake %	Real %
FF++	Train	3600	720	72	72
	Validation	700	140	14	14
	Test	700	140	14	14
	Total	5000	1000	100	100
Celeb-DF	Train	4281	390	75.92	66.10
	Validation	525	60	9.31	10.17
	Test	833	140	14.77	23.73
	Total	5639	590	100	100
DFD	Train	2205	247	71.87	68.04
	Validation	350	42	11.41	11.57
	Test	513	74	16.72	20.39
	Total	3068	363	100	100

**Fig. 2** I-frames extraction from an 8-second video

separate validation set ensures proper hyperparameter tuning and prevents overfitting to the training data, which further contributes to the model's robustness and generalizability.

The dataset is split in accordance with the most common approach used in state-of-the-art methods, specifically following the splitting ratio outlined in (Thing 2023). To maintain consistency, the source videos used to generate deepfakes were kept within the same split. As a result, the distribution percentages of real and fake videos differ slightly, but the minimum percentages mentioned above are maintained. The splits of the videos in each dataset are shown in Table 2.

3.2.2 Frame extraction

Videos consist of a sequence of images called frames. Frame extraction is a process of selecting individual frames from a video sequence. This work extracts keyframes, or I-frames, from real and fake videos. I-frames are important because they hold the most visual information and often mark significant changes or transitions in the video, such as movements, expressions, or events (referred to as "actions") (Wang et al. 2023). Analyzing I-frames helps detect subtle alterations that may not appear in other frames. The extraction process identifies I-frames and feeds them into the detection model, reducing computational load and improving the detection of deepfake artifacts. Collecting more I-frames enhances the accuracy and reliability of the system. Figure 2 shows an example of I-frame extraction from an 8-second video at 30 fps.

Table 3 Dataset splits after I-frames Extraction

Dataset	Set	Fake frames	Real frames	Fake %	Real %
FF++	Train	12492	1179	65.63	72.07
	Validation	2645	217	13.90	13.26
	Test	3897	240	20.47	14.67
	Total	19034	1636	100	100
Celeb-DF	Train	130101	11859	72.97	62.84
	Validation	18231	2074	10.22	10.99
	Test	29970	4939	16.81	26.17
	Total	178302	18872	100	100
DFD	Train	19870	820	74.53	68.79
	Validation	1249	143	4.68	12.00
	Test	5542	229	20.79	19.21
	Total	26661	1192	100	100

Table 4 Frames splits after balancing

Dataset	Set	Fake frames	Real frames	Fake %	Real %
FF++	Train	12492	11979	65.63	67.47
	Validation	2645	1897	13.90	10.68
	Test	3897	3880	20.47	21.85
	Total	19034	17756	100	100
Celeb-DF	Train	130101	127251	72.97	72.79
	Validation	18231	18171	10.22	10.39
	Test	29970	29408	16.81	16.82
	Total	178302	174830	100	100
DFD	Train	19870	16354	74.53	74.29
	Validation	1249	1064	4.68	4.83
	Test	5542	4597	20.79	20.88
	Total	26661	22015	100	100

Following the extraction of I-frames from all videos, Table 3 presents the total number of extracted frames for each dataset.

3.2.3 Frame Balancing

Frame balancing aims to ensure that the model is trained on an evenly distributed dataset, reducing bias toward any specific class. A noticeable imbalance between fake and real video classes is denoted in Table 3, where the fake class contains significantly more frames. To address this issue, more samples were added to the minority class (real videos) by extracting extra random frames. This balanced the number of real and fake frames, helping the model learn from an evenly distributed dataset and improving its ability to distinguish between real and fake content. To ensure the authenticity and diversity of real data, methods like data augmentation, oversampling, or synthetic data generation were avoided. These techniques can introduce artificial variations or artifacts (e.g., rotations, flips, or brightness adjustments) that do not accurately reflect real-world scenarios, which could affect the model performance. Instead, the real class was balanced by the extraction of additional frames from real videos, to maintain a natural distribution. The final frame distribution is shown in Table. 4.

3.2.4 Face region extraction

After extracting and balancing the frames, face detection is applied to isolate facial regions in each frame. This helps the model focus on the face, improving its ability to detect manipulations. The Dlib-HOG face detector (Dalal and Triggs 2005) is used due to its balance of accuracy and efficiency. It is an image-based method in which discriminative features are generally learned from training images and applied dimensionality reduction for efficiency (Kumar et al. 2019). Unlike Eigenfaces-based detection assumes that facial images lie in a linear subspace (Ho et al. 2024). This assumption often fails in real-world scenarios with variations in pose, lighting, and facial expressions (Ho et al. 2024). Image-based methods like Dlib-HOG handle these variations more effectively. Also, a recent study by Abdullah and Ali (2024) found that Dlib-HOG outperforms other methods, such as MTCNN, for face detection in deepfake detection.

Figure 3 below presents examples of faces extracted using the Dlib-HOG face detector from the frames of the video.

3.2.5 Face normalization and resizing

After face detection, the extracted facial regions undergo normalization and resizing to ensure uniformity across all input images. Normalization adjusts the pixel intensity values to a consistent range, typically between 0 and 1, which helps in stabilizing the training process and improving the model's convergence (Patel 2023).

Mathematically, normalization is performed by rescaling each pixel value pix of the image, which originally ranges from 0 to 255 (since pixel values in an 8-bit grayscale or

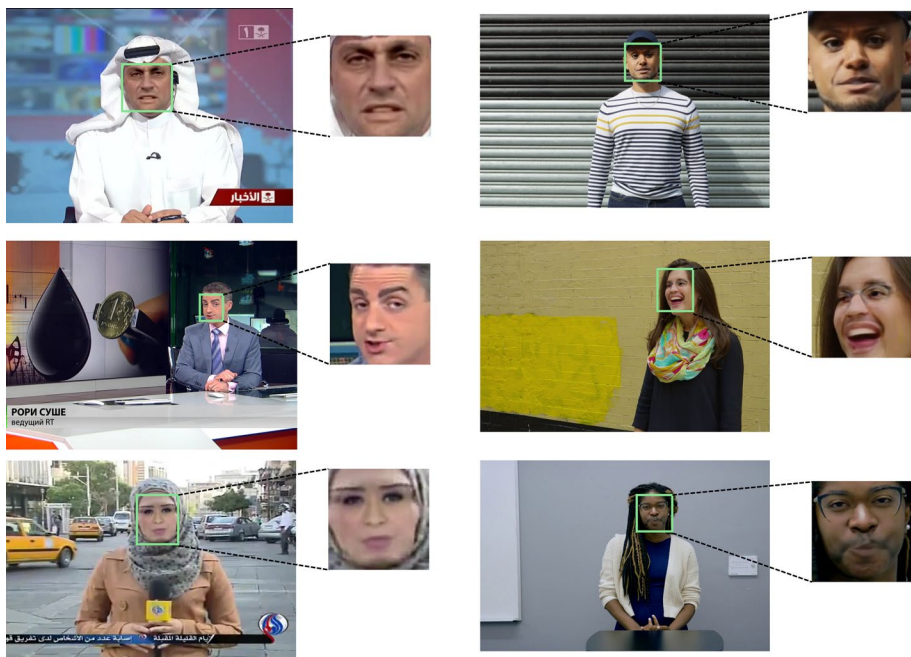


Fig. 3 Extracted faces using Dlib-HOG detector

RGB image are integers between 0 and 255) (Patel 2023). The normalization pix' is done by dividing each pixel value by 255, which can be expressed as:

$$pix' = \frac{pix}{255} \quad (1)$$

After normalization, resizing the faces to a fixed resolution ensures that the input dimensions are consistent with the model's requirements, typically 224×224 pixels for BEiT architectures (Bao et al. 2021). This step is important in standardizing the input data, allowing the model to learn from a homogeneous set of images, and ultimately improving its detection performance (Patel 2023).

3.3 The proposed deepfake detection model

After the completion of the data preprocessing pipeline, which ensures that high-quality, balanced, and facially focused frames are fed into the system. The next stage involves the building of a deepfake detection model. In this section, the proposed detection model is introduced, which is built upon a pre-trained self-supervised BEiT architecture (Bao et al. 2021).

3.3.1 Pre-trained Self-supervised BEiT

The BEiT model was first trained in a self-supervised way using the ImageNet-22k dataset. During this phase, the model learned to reconstruct missing parts of images, allowing it to understand image structures without needing labeled data. Afterward, it was fine-tuned for image classification using the labeled ImageNet-1k dataset, as illustrated in Fig. 4.

In this study, instead of training the BEiT model from scratch, the pre-trained weights from the fine-tuned ImageNet-1k model were used as the initial weights for the proposed BEiT-HPR model. This approach improves the performance of the training process because the model begins with learned features that help it understand the data better. It also speeds up training, as the model does not need to learn basic patterns from the beginning, allowing it to focus on task-specific features.

This approach is particularly advantageous for datasets and tasks where labeled data is limited, as the pre-trained model has already acquired a broad understanding of general image features from a large-scale dataset.

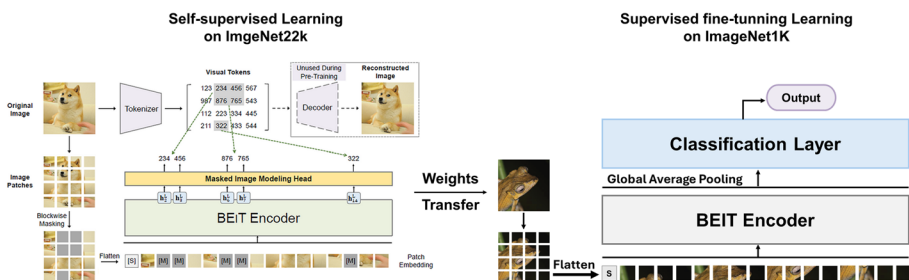


Fig. 4 Self-Supervised BEiT Model

3.3.2 The enhanced BEiT architecture: the BEiT-HPR model

The enhanced BEiT model represents a significant contribution to the advancement of vision transformer-based architectures that are known for their high computational complexity. The core concept revolves around enhancing the BEiT by strategically reducing the complexity of the model while maintaining the same performance. This is achieved through the introduction of a **PatchReducer Layer**. This layer is an enhanced version of the PatchMerger concept in (Renggli et al. 2022), which is inserted at specific positions within the BEiT encoder blocks. The PatchReducer is used to follow hierarchical structures seen in architectures like CNNs and the hierarchical vision transformer proposed in (Liu et al. 2021). Unlike the original BEiT, which uses a fixed number of image patches throughout all layers, this approach progressively reduces the number of patches. As a result, it mitigates the computational load while maintaining the same performance. Figure 5 shows the architecture of the BEiT with the Hierarchical PatchReducer (BEiT-HPR) model and details the PatchReducer Layer.

Following the same structure of the original BEiT illustrated in Fig. 4, the BEiT-HPR is built.

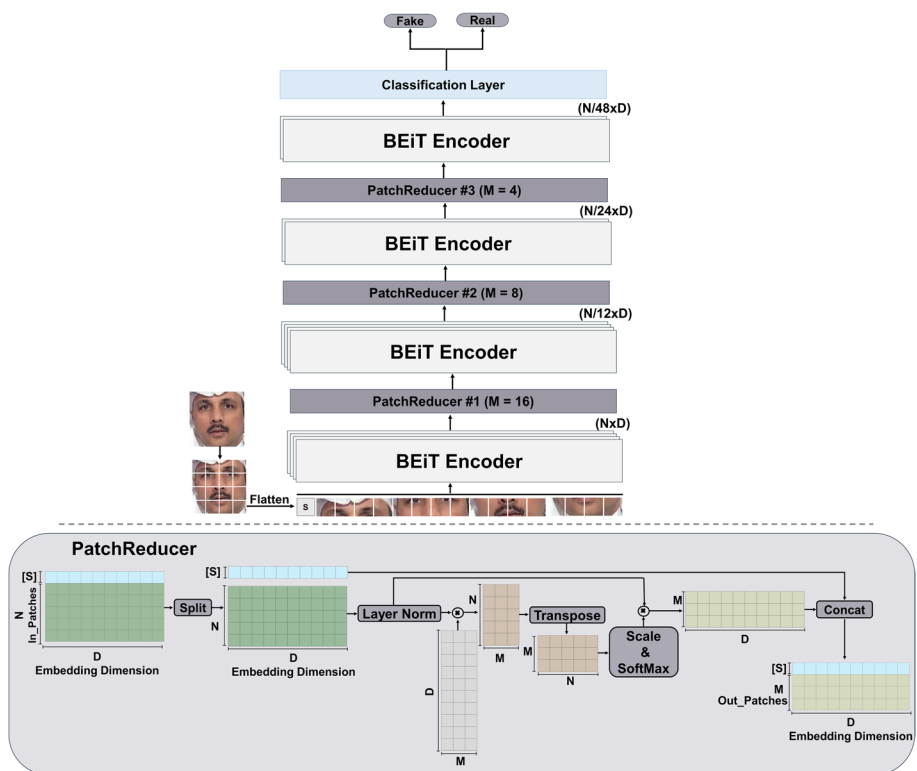


Fig. 5 The BEiT-HPR architecture. N: Number of Input Patches, [S]: Special Patch, D: Embedding Dimension Size, and M: Number of Output Patches

1. **Patch Embedding** The input image is first split into small patches. These patches are then flattened and converted into vectors for further processing. Also, a special patch, [S], is added to represent the entire image globally. This patch collects information from all patches throughout the model and is important for the final classification. Then, all these patches, along with [S], are embedded and passed to the encoder blocks.
2. **BEiT Encoder** The BEiT encoder is composed of alternating layers of Relative Multi-Head Self-Attention (RMHSA) (Bao et al. 2021) and Multilayer Perceptron (MLP) (Dosovitskiy et al. 2020) blocks. Layer Normalization (LN) (Lei Ba et al. 2016) is applied before each RMHSA and MLP layers, while LayerScale (LS) (Touvron et al. 2021) and residual connections are incorporated after each RMHSA and MLP layers. As shown in Fig. 5, the PatchReducer Layer is added between the encoder blocks in a hierarchy way. The first four encoder blocks maintain the original patch resolution with N patches. This is critical as the early layers are responsible for capturing fine-grained, low-level features such as edges, textures, and small details. After the first PatchReducer layer is applied, the number of patches is reduced by a factor of 12, resulting in $N/12$ patches. This occurs at the start of the fifth encoder block. These blocks focus more on mid-level abstractions, such as parts of faces or larger patterns. For $N = 196$ input patches, a PatchReducer layer with 16 output patches ($M = 16$) is applied after the fourth encoder block, effectively reducing the 196 patches by a factor of 12. The second PatchReducer layer further reduces the patch number by a factor of 24, leaving $N/24$ patches for the subsequent layers. At this depth, the network begins to focus on higher-level representations and global context. For $N = 196$ input patches, a PatchReducer layer with 8 output patches ($M = 8$) is applied after the eighth encoder block, effectively reducing the 196 patches by a factor of 24. The final PatchReducer layer is applied, reducing the number of patches to $N/48$. These final blocks deal with the most abstract representations, capturing the overall structure and semantics of the input image. For $N = 196$ input patches, a PatchReducer layer with 4 output patches ($M = 4$) is applied after the tenth encoder block, effectively reducing the 196 patches by a factor of 48. By progressively reducing the patches, the model balances performance and efficiency, focusing on the most meaningful features as it goes deeper into the network. More discussion about this hierarchy reduction is provided in Sect. 4.
3. **Classification Layer** At the end of the encoders, the [S] patch is extracted and passed to a classification layer. This layer is responsible for determining the authenticity of the input image, classifying it as either real or fake based on its features.

3.3.3 PatchReducer layer

PatchReducer Layer is a modified version of the one introduced by Renggli et al. (2022). The main modifications are done by incorporating learned initialization with Glorot uniform, applying scaled softmax for stability, and ensuring that the special patch [S] remains intact during the patch merging process.

If we represent the N input patches after layer normalization by a matrix $X \in \mathbb{R}^{N \times D}$, the merged representations $Y \in \mathbb{R}^{M \times D}$ are computed as follows:

$$Y = \text{softmax} \left(\frac{(XW)^T}{\sqrt{D}} \right) X \quad (2)$$

PatchReducer reduces the computational load by decreasing the number of patches processed, leading to notable speed improvements (40–60%) while preserving performance comparable to the original model. The following sections provide a detailed breakdown of each operation within the PatchReducer layer, as illustrated in Fig. 5.

1. **Splitting the Special Patch[S]** At the beginning of the PatchReducer process, the special token [S] (representing global information about the image) is separated from the main patch set, as shown in Fig. 5. This special token carries information about the entire input image and will be reintroduced after the processing of the patches.
2. **Layer Normalization** The remaining patches are passed through a Layer Normalization step, as illustrated in Fig. 5. This step ensures that the input patches are standardized in terms of scale and distribution. It helps the learned transformations in the following steps to be applied evenly across all patches, operating along the embedding dimension D . This process stabilizes training and improves performance by reducing internal covariate shifts.
3. **Multiplication by Learnable Matrix($\mathbb{R}^{D \times M}$)** After normalization, the patches are multiplied by a learnable matrix, as seen in Fig. 5, initialized with a Glorot uniform initializer. It projects the normalized patches into a new space. This matrix has dimensions $D \times M$, where M is the reduced number of patches, and D is the embedding dimension. The learnable matrix enables the model to determine how to combine the patches, deciding which spatial regions are most important for the next layer.
4. **Transpose Operation** The next step, shown in Fig. 5, involves transposing the resulting matrix. This reshapes the data to align the dimensions for matrix operations. Transposing prepares the model for an attention mechanism that evaluates the importance of each patch.
5. **Scale & Softmax Application** To combine patches, the transposed matrix is scaled by $\frac{1}{\sqrt{D}}$ before applying the softmax function, as depicted in Fig. 5. This scaling, similar to the scaled dot-product attention in transformers, prevents gradient vanishing or exploding, especially for large D . It stabilizes the softmax operation and improves training performance. The softmax function normalizes the patch values into probabilities that sum to 1. This helps the model focus on important patches by assigning them higher weights while reducing the impact of less relevant regions.
6. **Multiplication by Normalized Patches** After calculating the softmax weights, the model multiplies them with the original normalized patches, as shown in Fig. 5. This step merges the patches based on their importance, as determined by the softmax weights. The resulting patches provide a compact representation of the most important information from the original set.
7. **Concatenation with the Special Patch[S]** The reduced set of patches is then concatenated with the special patch [S], as shown in Fig. 5. This ensures that global information about the image is preserved for the next layers. Concatenating [S] allows the model to maintain a complete understanding of the image while reducing the number of patches for more efficient computation.

3.4 Complexity analysis

To examine the computational complexity of the proposed BEiT model, the number of floating-point operations (FLOPs) is used (Han et al. 2021). FLOPs is a hardware-independent metric that calculates the total arithmetic operations needed to train a model, where a higher FLOPs value reflects greater time complexity (Cong and Yang 2023). The following lines demonstrate the mathematical modeling used to calculate the total number of FLOPs in transformer-based models as stated in (Han et al. 2021).

Each transformer block has two main components: the multi-head self-attention mechanism and the MLP. The function $F(N, D)$, representing FLOPs, depends on the sequence length of patches N and the hidden dimension D , which is the size of the vector used to represent each patch. To compute the FLOPs associated with the multi-head self-attention mechanism, the projections of the query (q), key (k), and value (v) patches are taken into account. Accordingly, the FLOPs for these three projection operations are calculated as follows in Eq. 3, Eq. 4, and Eq. 5:

$$F_{qkv}(N, D) = 3ND^2 \quad (3)$$

$$F_A(N, D) = N^2D \quad (4)$$

$$F_{SA}(N, D) = N^2D \quad (5)$$

The FLOPs required for the linear projection of the concatenated self-attention output are:

$$F_\phi(N, D) = ND^2 \quad (6)$$

Thus, The overall computational FLOPs of the multi-head self-attention mechanism are outlined in Eq. 7 below:

$$\begin{aligned} F_{MSA}(N, D) &= F_{qkv}(N, D) + F_A(N, D) + F_{SA}(N, D) + F_\phi(N, D) \\ &= 3ND^2 + N^2D + N^2D + ND^2 \\ &= 4ND^2 + 2N^2D \end{aligned} \quad (7)$$

For the MLP, the FLOPs are determined by the fully connected layers. Specifically, projecting each patch requires $4ND^2$ FLOPs, leading to a total of $8ND^2$ FLOPs for the two fully connected layers in the MLP. Therefore, the total FLOPs for an MLP layer are given by:

$$F_{MLP} = F_{fc1}(N, D) + F_{fc2}(N, D) = 4ND^2 + 4ND^2 = 8ND^2 \quad (8)$$

The total number of FLOPs for a single transformer encoder (TE) is calculated using Eq. 9, including both the multi-head self-attention and MLP, which can be calculated by combining the FLOPs from the attention mechanism Eq. 7 and the MLP Eq. 8:

$$F_{TE} = F_{MSA}(N, D) + F_{MLP}(N, D) = 12ND^2 + 2N^2D \quad (9)$$

3.5 Experimental setup and parameter setting

The datasets were preprocessed using Python in the Jupyter Notebook IDE that is installed on a local machine equipped with a 64-bit Windows 11 Pro Operating System, an Intel Core i7 11th Generation Processor, 64 GB of RAM, and an Nvidia GeForce RTX 3050 Ti with 4 GB of GPU. Then, the models were trained using the Kaggle platform, which provided access to a powerful NVIDIA TESLA P100 with 16 GB of GPU.

The model was trained for a total of 100 epochs, with early stopping and a small initial learning rate to prevent overfitting. A callback mechanism was used, where training stopped if the validation loss did not improve after 10 epochs. The initial learning rate was set to 1×10^{-5} , and if no improvement in validation loss was observed, the learning rate was reduced by a factor of 0.5. If no further improvement occurred after an additional 5 epochs, the training was stopped. Table 5 below shows the parameters used in training the proposed BEiT-HPR model.

4 Sensitivity analysis

Sensitivity analysis explores how variations in model inputs influence its outputs. It provides insights into the relationships within complex systems across various domains. In this section, the focus is on analyzing the sensitivity of the BEiT-HPR model to several key factors. By systematically adjusting these components, the goal is to assess their impact on model accuracy and computational efficiency. The analysis primarily targets three critical aspects: Weights Initialization, Number of Output Patch, and the Position of the PatchReducer layer, evaluating their role in shaping the model’s overall performance.

4.1 Weights initialization

Weight initialization involves setting the initial values of parameters in neural network models before training them on a dataset. In this work, for the original BEiT model, three initialization strategies were evaluated: (1) Random Normal Initialization, where all weights are initialized randomly using normal distribution and the model is trained from scratch, (2) Partial Fine-Tuning the BEiT Model, where ImageNet21k-ft1k (weights from self-super-

Table 5 Summary of Training Parameters

Parameter	Parameter setting
Image Size	$224 \times 224 \times 3$
Batch size	16
Epochs	100
Optimizer	Stochastic Gradient Descent (SGD)
Loss function	Binary Cross-Entropy Loss
Learning rate	Initial Learning Rate: 1×10^{-5} Monitor: Validation Loss Patience: 5 Reducing Factor: 0.5 Minimum Learning Rate: 1×10^{-7}
Early stopping	Monitor: Validation Loss Patience: 10

vised pretraining on ImageNet-22k and fine-tuned with supervised on ImageNet-1k) pre-trained weights are used, but only the final layers are fine-tuned while earlier layers remain frozen, and (3) Full Fine-Tuning the BEiT Model, where ImageNet21k-ft1k pre-trained weights are used, and all layers of the model are fine-tuned on the target dataset. Table 6 below summarizes the accuracy achieved with each initialization strategy on DFD dataset.

Referring to Table 6, when initializing all layers with random weights, the model learns distinguishing features from scratch. Training from scratch generally demands larger datasets and more computational resources, as the model lacks pre-learned features. In this analysis, random initialization resulted in a model accuracy of 71.80%, indicating limited effectiveness in feature learning for the task of detecting fake faces with the used dataset.

In the partial fine-tuning with pre-trained weights configuration, the model is initialized with ImageNet21k-ft1k weights. However, only the final layers are fine-tuned, while the earlier layers remain frozen. Freezing the earlier layers means setting the weights of those layers as non-trainable during the training process. These layers are not updated with back-propagation and their weights remain fixed. This strategy preserves general features learned from the ImageNet dataset while modifying the final layers to suit another particular classification purpose. This configuration resulted in an accuracy of 85.00%, suggesting that while the pre-trained features are valuable, freezing the earlier layers restricts the model's ability to fully specialize for detecting subtle artifacts in fake faces.

The full fine-tuning with pre-trained weights approach initializes the model with ImageNet21k-ft1k weights and fine-tunes all layers, enabling the model to leverage prior knowledge of general image features such as edges, textures, and shapes. Fine-tuning all layers allows these pre-trained features to be adapted specifically for the deepfake detection task, resulting in an observed accuracy of 98.20%, the highest among the tested configurations. This outcome highlights the benefits of transfer learning and demonstrates the effectiveness of self-learning weights in improving the model's performance for deepfake detection.

4.2 Output patch selection

In the original PatchMerger layer added to the standard ViT model, Renggli et al. (2022) recommended outputting 8 patches midway through the transformer encoder block sequence. In this analysis, the proposed PatchReducer layer was evaluated using various output patch sizes. This evaluation examines their impact on model accuracy, computational complexity (measured in FLOPs using Eq. 9), and training time, as shown in Table 7. The PatchReducer layer was positioned after the sixth encoder block, which is the midpoint of the 12-block encoder stack in the BEiT model, allowing for a controlled comparison across different output patch configurations.

The original BEiT model outputs a fixed number of N patches. In the standard base BEiT the number of patches is 196 (Bao et al. 2021). This configuration achieves a high accuracy of 98.20%. However, this configuration also leads to substantial computational complexity (17.6 billion FLOPs) and extended training time (6.1 h). Reducing the number of output patches slightly decreases accuracy but reduces computational requirements by over 40%

Table 6 Accuracy results for different weight initialization strategies

Weights initialization strategy	Accuracy
Random Normal Initialization	71.80%
Partial Fine-Tuning the BEiT Model	85.00%
Full Fine-Tuning the BEiT Model	98.20%

Table 7 Effect of Output Patch Selection on Performance, Complexity, and Training Time

Output patches size	Accuracy (%)	FLOPs (Giga)	FLOPs Reduction Rate (%)	Training time (Hours)	Speed-up time rate (%)
4	97.97	9.0850	48.45	2.00	67.21
8	98.02	9.3000	47.23	2.31	62.13
16	98.06	9.6020	45.52	2.22	63.61
32	98.10	10.2086	42.08	2.30	62.30
64	98.13	11.6973	33.63	3.75	38.52
196 (Original)	98.20	17.6245	-	6.10	-

and speeds up training time by more than 60% for patch sizes smaller than 64, as shown in Table 7. For instance, with 8 output patches, the model attains an accuracy of 98.02%, along with reduced FLOPs (9.3 billion) and a faster training time (2.31 h).

This comparison shows that selecting fewer output patches, such as 4, 8, or 16, offers an effective balance between model accuracy and efficiency. This approach presents an effective alternative to the original configuration, especially for applications with limited computational resources.

4.3 PatchReducer layer position

This subsection examines the effect of varying the position of the PatchReducer layer on model performance. As highlighted in the previous section 4.2 and shown in Table 7, reducing the output to 4, 8, or 16 patches provides a favorable balance between model complexity (maintaining FLOPs below 10 billion) and model performance, achieving approximately 98% accuracy. To further analyze this, the PatchReducer layer was positioned after different encoder blocks within the model, and its impact was evaluated on configurations with 4, 8, and 16 output patches.

Figure 6 shows the accuracy and FLOPs results for placing the PatchReducer layer after each encoder block, with outputs of 4, 8, and 16 patches. The patterns for 16, 8, and 4 patches are similar in terms of accuracy and FLOPs. The results indicate that placing the PatchReducer layer deeper in the encoder generally improves accuracy but increases FLOPs and training time.

In the early blocks, reducing patches too much can remove features that might become important later in the model. To address this, the hierarchical PatchReducer layers are arranged as shown in Fig. 5. Keeping 16 patches after the first four blocks helps the model capture detailed features, improving performance. After block 8, accuracy differences between configurations are smaller, making it a good position to reduce to 8 patches and lower the computational cost. In the later stages, accuracy stays stable even with fewer patches, so reducing to 4 patches after block 10 minimizes computation while keeping important global features.

This approach balances accuracy and efficiency, allowing the model to reduce computational load gradually while preserving key features at each stage.

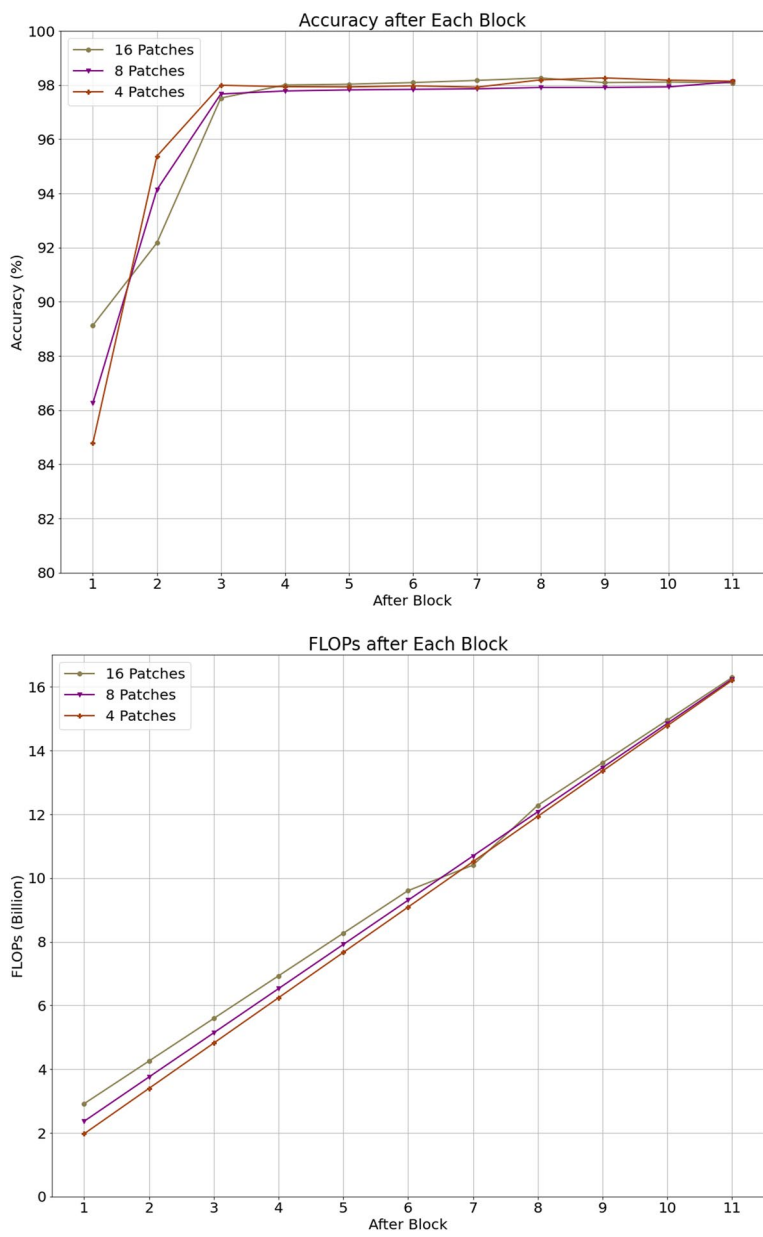
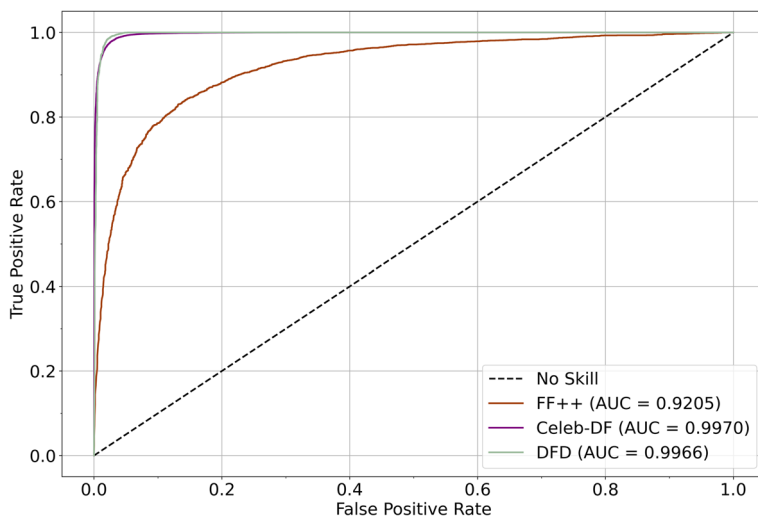


Fig. 6 Effect of Positioning 16, 8, and 4 Patches

Table 8 Performance metrics (PR: Precision, RC: Recall, F1: F1-score, Accuracy, and Cross-Entropy Loss) of the proposed BEiT-HPR model on the FF++, Celeb-DF, and DFD datasets

Class	FF++			Celeb-DF			DFD		
	PR	RC	F1	PR	RC	F1	PR	RC	F1
Real	87.72	78.90	83.08	97.79	97.33	97.56	98.84	97.25	98.04
Fake	80.81	88.95	84.69	97.39	97.85	97.61	97.77	99.06	98.41
Weighted Avg	84.27	83.92	83.88	97.59	97.59	97.59	98.25	98.25	98.24
Accuracy	83.92%			97.59%			98.25%		
Testing Cross-Entropy Loss	0.5595			0.0815			0.0658		

**Fig. 7** AUC-ROC for each dataset

5 Results and discussion

This section presents the experimental results obtained from evaluating the BEiT-HPR model across both in-data and cross-data scenarios.

5.1 In-data evaluation

In the in-data evaluation, models are trained and tested on the same dataset to assess their ability to learn and generalize within a consistent data distribution. This evaluation is conducted separately on the FF++, Celeb-DF, and DFD datasets.

Table 8 below summarizes the evaluation metrics of the BEiT-HPR model on each dataset. Additionally, the performance for each dataset is visualized using the AUC-ROC curve in Fig. 7.

For the FF++ dataset, as shown in Table 8, the model achieved a precision of 87.72% for real class and 80.81% for fake class. This indicates that the model tends to be more conservative when classifying real faces, possibly at the cost of some false positives in the fake class. However, the recall for the real class was substantially lower (78.90%) than

for the fake class (88.95%), indicating that the model is better at correctly identifying fake instances than real ones. Resulting in potentially a higher false positive rate for the real class. The model demonstrated almost balanced values of F1-score for both classes with 83.08% and 84.69% for real and fake classes, respectively. The overall accuracy of 83.92% reflects a good balance between precision and recall. The Cross-Entropy Loss is relatively significant 0.5595, indicating that the model's predictions are less confident or more uncertain on the FF++ dataset. This is due to the lower video quality, compression artifacts, lighting, and manipulation techniques in the dataset. These factors make it harder for the model to distinguish real from fake content, leading to increased uncertainty in predictions. The AUC-ROC curve for this dataset is visualized in Fig. 7, and the calculated AUC score is 0.9202. This score further highlights the model's reasonable ability to distinguish between real and fake images, though there is room for improvement compared to the other datasets.

For the Celeb-DF dataset, the model performed exceptionally well, as indicated in Table 8, achieving high precision and recall for both real (97.79% precision, 97.33% recall) and fake (97.39% precision, 97.85% recall) classes. The weighted average F1-score remained consistent at 97.59%, with a high F1-score of 97.56% for real images and 97.61% for fake images. The model showed its strength in distinguishing between real and fake images on this dataset, as reflected in the high accuracy of 97.59%. The Cross-Entropy Loss is low 0.0815, implying that the model is confident in its predictions on this dataset compared to the FF++ dataset. The AUC-ROC curve for this dataset is visualized in Fig. 7, and the AUC score is 0.9970. This nearly perfect score demonstrates the robustness of the proposed method on this challenging dataset as its fake videos are generated using an enhanced deepfake synthesis technique. These results demonstrate that the BEiT-HPR model is highly effective in detecting deepfakes with strong confidence.

On the DFD dataset, as seen in Table 8, the precision for real images was 98.84%, and recall was 97.25%, while for fake images, precision was 97.77% and recall was 99.06%. These results show excellent detection performance, with the model excelling in both distinguishing real from fake images and maintaining a high level of precision and recall across both classes. Also, the model achieved F1-scores of 98.04% for real images and 98.41% for fake images, with a weighted average F1-score of 98.24%. The overall accuracy of 98.25% further emphasizes the effectiveness of the proposed BEiT-HPR model in detecting deepfakes in the DFD dataset. Also, the BEiT-HPR model achieved the lowest cross-entropy loss of 0.0658, indicating extremely confident predictions with minimal uncertainty on this dataset. The AUC-ROC curve for this dataset is shown in Fig. 7, with an AUC score of 0.9966. This high score confirms the model's reliability and precision on the DFD dataset, demonstrating its strong performance across diverse deepfake scenarios.

In summary, the model demonstrated strong performance across all datasets, with particularly impressive results on Celeb-DF and DFD, as evidenced by their near-perfect AUC scores of 0.9970 and 0.9966, respectively. The relatively high recall values, especially for the fake class, indicate that the model can identify a large proportion of deepfake images while maintaining good precision. The AUC-ROC curves provide further validation of the model's robustness, highlighting its capability to differentiate between real and fake images effectively.

To obtain a more detailed understanding of the model's performance, the confusion matrices are utilized to provide an in-depth examination of the model's predictions. Figure 8

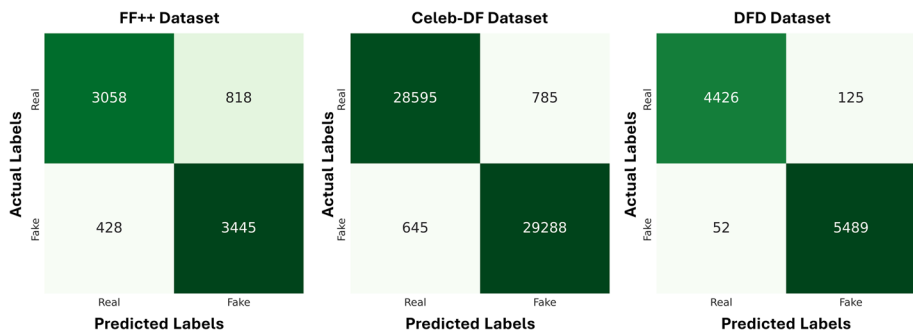


Fig. 8 Confusion matrices for each dataset

presents the confusion matrix for each dataset, where 'Fake' is considered the positive class and 'Real' is the negative class.

As shown the the first matrix in Fig. 8, for the FF++ dataset, the model correctly identified 3445 fake samples (true positives) and 3058 real samples (true negatives). However, it misclassified 818 real samples as fake (false positives) and 428 fake samples as real (false negatives). The false positive rate is higher, showing that the model struggles to distinguish between real and fake samples.

Moving to the second matrix for the Celeb-DF dataset, the model showed strong performance. It correctly identified 29,288 fake samples (true positives) and 28,595 real samples (true negatives). The number of misclassified samples was low, with 785 real samples classified as fake (false positives) and 645 fake samples classified as real (false negatives). The model performs very well on this dataset, with high precision and recall for fake detection, as shown before in Table 8

Lastly, the model performed almost perfectly for the DFD dataset. It correctly identified 5489 fake samples (true positives) and 4426 real samples (true negatives). Only 125 real samples were classified as fake (false positives), and 52 fake samples were classified as real (false negatives). The model's ability to distinguish between real and fake samples in this dataset is excellent.

5.2 Cross-data evaluation

In cross-data evaluation, the model's generalization ability is tested by training on one dataset and then evaluating on another separate unseen dataset. This approach simulates real-world conditions where the test data may differ significantly from the training data. Table 9 provides a summary of the results from the cross-data evaluation.

When trained on the FF++ dataset, the model struggled to generalize to Celeb-DF and DFD. Specifically, performance dropped to an accuracy of 49.88% and an F1-score of 46.14 on Celeb-DF, while achieving 63.98% accuracy and an F1-score of 60.38 on DFD. These results highlight the challenges of adapting to datasets with different manipulation techniques, video compression, lighting and quality levels when trained only on FF++, largely due to differences in quality and characteristics between FF++ videos and those in Celeb-DF and DFD, such as varying types of fake videos and resolutions.

Table 9 cross-data evaluation results

Testset →	FF++			Celeb-DF			DFD		
Trainset ↓	ACC	PR	RC	F1	ACC	PR	RC	F1	F1
F++	83.92	84.27	83.92	83.88	49.88	49.50	49.88	46.14	60.38
Celeb-DF	55.37	58.82	55.37	50.53	97.59	97.59	97.59	97.59	61.86
DFD	51.79	52.86	51.79	46.88	82.33	83.42	82.33	82.17	98.24

Training the model on the Celeb-DF dataset yielded moderate performance on DFD, with an accuracy of 64.56% and an F1-score of 61.86. However, adaptation to FF++ proved more difficult, resulting in an accuracy of only 55.37% and an F1-score of 50.53. This suggests that while Celeb-DF provides challenging deepfake scenarios for training, its specific characteristics may limit generalizability to datasets with simpler manipulation methods, such as FF++.

Using the DFD dataset for training produced similar trends. While the model performed very well on Celeb-DF, achieving 82.33% accuracy and an F1-score of 82.17, its performance on FF++ was limited, with an accuracy of 51.79% and an F1-score of 46.88. This highlights the challenges posed by datasets with diverse manipulations when training data lacks variety.

In summary, despite the model's limited performance in cross-data evaluation, the choice of training dataset significantly influenced the results. Notably, training on DFD led to excellent performance on Celeb-DF, and moderate performance was achieved on DFD when trained on either FF++ or Celeb-DF. These findings suggest that the model is affected by some dataset characteristics, such as differences in video quality, compression, or type of fakeness.

6 Ablation study of patchreducer layer

The ablation study examines the impact of the PatchReducer layer on the performance of the BEiT-HPR model. By comparing results with and without the PatchReducer layer across each dataset, this analysis highlights its contribution to both detection accuracy and computational efficiency.

Table 10 summarizes the ablation study on the complexity of the PatchReducer layer. The complexity of each model is measured in FLOPs (floating-point operations), calculated using the method described in section 3.4 and Eq. 9. Additionally, the table analyzes the number of positional embedding parameters (Pos_Embeddings) affected by the PatchReducer layer.

The original BEiT architecture demonstrates a computational complexity of 17.6245 billion FLOPs, reflecting the high resource demand typically associated with transformer-based models. In contrast, the proposed BEiT-HPR, which incorporates the PatchReducer layer at strategic encoder stages, achieves a drastic reduction in complexity to 6.6553 billion FLOPs. This represents an approximately 62.2% decrease in computational demand while maintaining competitive detection performance, as discussed in previous sections.

In addition, the proposed BEiT-HPR model reduces positional embedding (Pos_Embeddings) parameters by 63.4%, from 105,408 to 38,592. Positional embeddings are directly tied to the number of patches, unlike other parameters linked to the model's architecture and hidden dimensions. This reduction improves parameter efficiency while preserving the model's ability to encode spatial information.

The large decrease in FLOPs and number of positional embedding parameters highlights how effectively the PatchReducer layer simplifies the model's operations. By reducing patches in the later stages of the encoder, the BEiT-HPR minimizes the size of the data being processed, which lowers the number of calculations needed.

Table 10 Ablation Study of PatchReducer Layer on Complexity

Architecture	FLOPs (Billion)	FLOPs reduc- tion rate	Pos_em- beddings parameters	Pos_em- beddings parameters reduction rate
BEiT	17.6245	62.2%	105, 408	63.4%
BEiT-HPR	6.6553		38, 592	

Table 11 Ablation Study of PatchReducer on Performance and Training Time

Dataset	Architecture	Accuracy	Training time per epoch	Speed- up rate
FF++	BEiT	85.35%	11 min 65 sec	55.1%
	BEiT-HPR	83.92%	5 min 26 sec	
Celeb-DF	BEiT	98.14%	1 hr 46 min 32 sec	51.3%
	BEiT-HPR	97.59%	51 min 51 sec	
DFD	BEiT	98.08%	15 min 11 sec	51.1%
	BEiT-HPR	98.25%	7 min 26 sec	

Moreover, Table 11 presents the results of the ablation study on the PatchReducer layer's impact on model performance and training time. This analysis compares the original BEiT architecture with the BEiT-HPR model, which incorporates the PatchReducer layer. The evaluation is conducted across the three datasets: FF++, Celeb-DF, and DFD.

Regarding performance, the results in Table 11 show that the BEiT-HPR generally maintains or slightly improves the accuracy compared to the BEiT model. For the FF++ dataset, BEiT achieves an accuracy of 85.35%, while the BEiT-HPR model delivers a slightly lower accuracy of 83.92%. On Celeb-DF, BEiT reaches 98.14%, while the BEiT-HPR performs slightly less at 97.59%. However, the BEiT-HPR model shows a notable improvement on the DFD dataset, achieving 98.25% accuracy, compared to BEiT's 98.08%. These results demonstrate that the PatchReducer layer does not significantly harm the model's performance and in some cases can enhance it, especially in datasets like DFD.

In terms of training time, the BEiT-HPR model offers a clear advantage. For FF++, the training time per epoch is reduced by 55.1%, from 11 min and 65 s with BEiT to 5 min and 26 s with BEiT-HPR. On Celeb-DF, the reduction is even more remarkable, with BEiT requiring 1 h, 46 min, and 32 s per epoch, compared to 51 min and 51 s with BEiT-HPR, a difference of 54 min and 41 s, meaning it speeds up the training time by more than 50%. For DFD, BEiT takes 15 min and 11 s per epoch, while BEiT-HPR reduces this by 51.1%, taking only 7 min and 26 s. These highly significant reductions in training time highlight the efficiency gains provided by the PatchReducer layer, which effectively reduces computational complexity during training.

In summary, the ablation study demonstrates that the BEiT-HPR model, while maintaining competitive or improved accuracy, offers substantial reductions in both training time and FLOPs across all datasets. This improvement makes the model more efficient and practical without losing performance. It is significant to highlight that, reducing the complexity of the model not only accelerates training time but also reduces the physical requirements for deploying the model. This translates to lower energy consumption, reduced demand for high-performance hardware, and overall cost savings in computational resources. These

advantages make the BEiT-HPR model a more practical and sustainable solution for deepfake detection, particularly in resource-limited environments.

7 Comparative evaluation

This section provides a comprehensive analysis of the performance of the proposed BEiT-HPR model in relation to other deep learning models, including state-of-the-art methods. The comparative study highlights the strengths of the approach by assessing its effectiveness in addressing the primary challenges associated with transformer-based models in deepfake detection.

7.1 Comparison with Other deep learning models

To evaluate the effectiveness of the proposed BEiT-HPR model, we implemented several well-known deep learning models and compared their performance against the BEiT-HPR. This comparison including DenseNet (Huang et al. 2017), XceptionNet (Chollet 2017), and ViT (Dosovitskiy et al. 2020). The presented results for all models are obtained using the same experimental settings, ensuring a fair and consistent evaluation across all models. Moreover, these models were also trained using the same weight initialization strategy as the BEiT-HPR, where the pre-trained weights from the ImageNet dataset were utilized. Then, all layers were subsequently fine-tuned during training to optimize performance.

Table 12 presents a comparison of the BEiT-HPR model with DenseNet, XceptionNet, and ViT across three datasets (FF++, Celeb-DF, and DFD), using accuracy as the evaluation metric. The analysis of the results is conducted in both in-data and cross-data settings to emphasize the performance differences.

Considering training on FF++, the BEiT-HPR model achieved the highest accuracy of 83.92% when tested on FF++, outperforming DenseNet, XceptionNet, and ViT, which scored 74.36%, 77.82%, and 79.82%, respectively. However, in cross-data evaluations, BEiT-HPR's accuracy on Celeb-DF and DFD (49.88% and 63.98%, respectively) was lower than ViT, which obtained the top cross-data accuracy of 61.99% on Celeb-DF and 70.00% on DFD.

Table 12 Performance Comparison of BEiT-HPR with other deep learning models across different training and testing datasets. The highest accuracy for each testing set is highlighted in bold

Training set	Model	Accuracy of Testing Set (%)		
		FF++	Celeb-DF	DFD
FF++	DenseNet	74.36	52.90	59.46
	XceptionNet	77.82	56.45	65.08
	ViT	79.82	61.99	70.00
	BEiT-HPR	83.92	49.88	63.98
Celeb-DF	DenseNet	53.97	85.78	53.54
	XceptionNet	54.99	93.42	53.54
	ViT	54.34	94.30	54.34
	BEiT-HPR	55.37	97.59	64.56
DFD	DenseNet	51.34	53.04	91.14
	XceptionNet	56.37	68.53	92.58
	ViT	51.72	53.96	94.77
	BEiT-HPR	51.79	82.33	98.25

Analyzing the Celeb-DF training set, the BEiT-HPR model demonstrated exceptional performance in both in-data and cross-data evaluations. It achieved an impressive 97.59% accuracy when tested on Celeb-DF, surpassing ViT (94.30%), XceptionNet (93.42%), and DenseNet (85.78%). Additionally, it achieved a noteworthy cross-data accuracy of 64.56% on DFD, outperforming the other models.

For the DFD training set, BEiT-HPR once again outperformed its counterparts in the in-data evaluation, achieving the highest accuracy of 98.25%. This score is significantly higher than ViT (94.77%), XceptionNet (92.58%), and DenseNet (91.14%). It should be highlighted that when testing on the FF++ set, the proposed BEiT-HPR model achieved similar detection accuracy to the other deep learning models. Furthermore, it demonstrated the strongest cross-data performance for Celeb-DF, achieving 82.33% accuracy, exceeding the other models' cross-data results when trained on DFD.

In summary, the results in Table 12 show that transformer-based models outperform CNN-based models in in-data evaluation. Among these, the proposed BEiT-HPR model achieves the best performance, surpassing the ViT model. BEiT-HPR demonstrates the highest detection accuracy on the Celeb-DF and DFD datasets. However, the model's performance is influenced by the types of fakeness present in the video. For example, it struggles to maintain high accuracy on the FF++ dataset due to its diverse fakeness types. Moreover, all the tested models face challenges in cross-domain testing due to varying video quality across datasets. Celeb-DF and DFD generally have higher video quality with minimal compression artifacts, while the FF++ (C23) dataset shows more significant compression artifacts. However, BEiT-HPR demonstrates exceptional cross-domain performance between DFD and Celeb-DF compared to FF++ and other datasets. This suggests that BEiT-HPR is sensitive to video quality and performs better with high-quality videos that have low compression.

The complexity of deep learning models is also analyzed in addition to their performance comparison. Table 13 compares the computational complexity of the proposed BEiT-HPR with the mentioned deep learning models. DenseNet has 2.8859 billion FLOPs, and XceptionNet has 4.5810 billion FLOPs, reflecting the common observation that CNN-based models generally have lower complexity than transformer-based models. ViT, for example, shows the highest complexity at 17.6215 billion FLOPs. Notably, there is a lack of existing work in the literature that explicitly addresses the computational complexity of deep learning models for deepfake detection. The only work that discusses complexity in this domain is Shallow ViT by Usmani et al. (2024), which aims to reduce the computational complexity by changing the image shape, batch size, number of attention heads, and linear projection dimensions. Their settings adjusting reduced the FLOPs to 7.8 billion.

Despite the lower complexity of CNN-based models, BEiT-HPR outperforms them, highlighting its efficiency and effectiveness. Compared to XceptionNet, BEiT-HPR has slightly higher complexity (about 2 billion more FLOPs) but achieves better performance, justifying the additional cost. Furthermore, BEiT-HPR is significantly less complex than ViT, reducing complexity by over 60%, making it a more efficient yet powerful transformer-based model.

Table 13 Complexity Comparison of BEiT-HPR with other deep learning models

Model	FLOPs (Billion)
DenseNet	2.8859
XceptionNet	4.5810
ViT	17.6215
Shallow ViT (Usmani et al. 2024)	7.8
BEiT-HPR	6.6553

In summary, the BEiT-HPR model excels in in-data evaluations, delivering the highest accuracy on FF++, Celeb-DF, and DFD when trained on their respective datasets. In cross-data evaluations, it surpassed other models in specific scenarios, particularly when trained on Celeb-DF or DFD, though it showed less strength when trained on FF++. In terms of complexity, the BEiT-HPR model offers a significant reduction in computational cost compared to other transformer-based models, reducing the number of positional embedding parameters and overall FLOPs. While its complexity is slightly higher than some CNN-based models, such as XceptionNet, it outperforms them in terms of detection accuracy. Furthermore, BEiT-HPR is much less complex than models like ViT. This analysis underscores the BEiT-HPR model's effectiveness and efficiency in deepfake detection, making it a highly efficient solution that maintains high performance.

7.2 Comparison with existing state-of-the-art models

In this section, a performance comparison is presented between the proposed BEiT-HPR model and several existing deepfake detection models. Table 14 below highlights the performance of various models across different datasets, including FF++, Celeb-DF, and DFD. This comparison evaluates the effectiveness of the proposed approach compared to existing techniques, focusing only on detection performance (accuracy or AUC scores). It is important to note that the comparison is limited to accuracy since the other works did not include any time complexity measurements. The bold values in the table indicate models that have achieved lower performance than the proposed BEiT-HPR model, demonstrating the improvements offered in deepfake detection.

In the comparison of models on the FF++ dataset, the proposed BEiT-HPR model demonstrates competitive performance with an accuracy of 83.9%. While this is not the highest among all models, it outperforms several others. For instance, the DINO model, which achieves an accuracy of 81.85%, falls behind the BEiT-HPR model, indicating that the proposed approach is more effective in deepfake detection for this dataset. Additionally, the base BEiT model, with an accuracy of 86.82%, slightly outperforms the BEiT-HPR model, suggesting that the modifications in BEiT-HPR, while successful in reducing computational complexity, might have a small trade-off in performance.

However, when compared to some of the top-performing models, the BEiT-HPR model does not reach their level of accuracy. It is important to note that some studies treat each fake category of the FF++ dataset (FS, F2F, NT, DF, and FSh) as separate datasets, training each fake category against the real class individually and then averaging the classification results of all fake categories to produce the final fake class classification. In contrast, other models, including the BEiT-HPR approach presented here, combine all the categories into a single fake class against the real class. This difference in approach may account for some discrepancies in performance between models. However, the works listed in the table do not specify whether they averaged the results of individual categories or treated them as one combined fake class.

As mentioned before, the data splitting method used in this work follows the approach outlined by Thing (2023), and the obtained results align closely with his findings. This indicates that, while the BEiT-HPR model may not outperform the most complex models, it performs robustly within the defined methodology and remains a strong contender in the deepfake detection landscape, especially considering its computational efficiency.

Table 14 Performance Comparison of the proposed BEiT-HPR model with existing deepfake detection models. The bold values highlight models with lower performance than the proposed approach

Study	Performance of testing set (%)		
	FF++	Celeb-DF	DFD
Three-layered FFNN 2021	-	-	86
Four-layered FFNN 2021	92.48	-	88.35
iCaps-Dfake 2021	-	86 (AUC)	-
Re-attention ViT 2022	98.18	99.31	-
Fusion of Xception, CapNet & DSP-FWA 2022	-	99.04 (AUC)	-
Inception-v3 SiamNet 2023	-	98.3	96.08
Two-stream XceptionNet 2023	-	97.47	-
Long Distance Attention Xception-Net 2023	-	99.13	-
Multi-branch XceptionNet with Global Attention 2023	-	98.37	-
iCapsNet-TSF 2023	-	94.07	-
BEiT 2023	86.82	87.51	99.33
Capsule GNN 2023	94.92	-	97.76
Interpretable Spatial-Temporal Video Transformer 2023	-	99.8	-
Spatial, Temporal & Spatiotemporal Transformer 2023	94.15 (AUC)	96.24 (AUC)	-
DINO 2023	81.85	88.90	-
MAE ConvNeXt with Multiscale ViT 2023	-	92.82	-
Spatio-temporal Transformer 2024	95.7	-	-
CoAtNet 2024	-	97.81	-
SWIN 2024	95.72	97.91	-
ViViT 2024	90.27	87.17	-
CNN-LSTM 2024	-	86	-
SegCaps-CNN + CapNet 2024	-	98.9	-
Self-Supervised Graph Conv Transformer 2024	-	99.47	-
The Proposed BEiT-HPR	83.9	97.59	98.25

The comparison results for the Celeb-DF dataset showed that the following models demonstrate lower performance than the BEiT-HPR model. First, Two-stream XceptionNet achieved 97.47% (Wang et al. [2023](#)). Similarly, iCapsNet-TSF reached 94.07% (Lu et al. [2023](#)). Additionally, iCaps-Dfake achieved 86% AUC (Khalil et al. [2021](#)). Furthermore, BEiT obtained 87.51% (Thing [2023](#)), while ViViT scored 87.17% (Ramadhani et al. [2024](#)). Moreover, DINO reached 88.90% (Khan and Dang-Nguyen [2023](#)), and CNN-LSTM achieved 86% (Mirzayev and DİBEKLİOĞLU [2024](#)). Lastly, MAE ConvNeXt with Multiscale ViT performed at 92.82% (Kavthekar et al. [2023](#)).

In particular, the iCapsNet-TSF (94.07%) (Lu et al. [2023](#)) and BEiT (87.51%) (Thing [2023](#)) models display a significant performance gap. These models show lower detection accuracy, indicating that their architectures are less effective for deepfake detection on the Celeb-DF dataset. The ViViT (87.17%) (Ramadhani et al. [2024](#)), DINO (88.90%) (Khan and Dang-Nguyen [2023](#)), CNN-LSTM (86%) (Mirzayev and DİBEKLİOĞLU [2024](#)), and MAE ConvNeXt with Multiscale ViT (92.82%) (Kavthekar et al. [2023](#)) also underperform when

compared to the BEiT-HPR model, reinforcing the effectiveness of the proposed approach over these existing methods.

The DFD dataset is one of the least used datasets in the literature, which makes it an important case for comparison. Among the models tested on this dataset, the BEiT-HPR model outperforms several others with a score of 98.25%. Several models show lower performance than the BEiT-HPR model, including the Three-layered FFNN (86%) (Sahla Habeeba et al. 2021), the Four-layered FFNN (88.35%) (Demir and Ciftci 2021), and Inception-v3 SiamNet (96.08%) (Kingra et al. 2023). Additionally, Capsule GNN achieved 97.76% (Venkatachalam et al. 2023), which is still below the BEiT-HPR performance.

The Three-layered FFNN (86%) (Sahla Habeeba et al. 2021) and Four-layered FFNN (88.35%) (Demir and Ciftci 2021) show a significant performance gap compared to the BEiT-HPR model. In addition, the Inception-v3 SiamNet (96.08%) (Kingra et al. 2023) and Capsule GNN (97.76%) (Venkatachalam et al. 2023) also demonstrate lower accuracy. These results highlight that the BEiT-HPR model offers a more effective solution for deepfake detection on the DFD dataset.

In summary, while the proposed BEiT-HPR model surpasses some existing methods in terms of detection accuracy, other works have achieved even higher performance. However, the main objective of this study is to provide a solution with reduced complexity while still achieving reasonable high detection accuracy. Many of the top-performing models, such as iCapsNet-TSF (Lu et al. 2023) and Self-Supervised Graph Conv Transformer (Khormali and Yuan 2024), rely on hybrid or more complex architectures. These models often combine multiple networks or advanced techniques to improve performance, making them more computationally demanding.

8 Conclusion

This work introduced an efficient and scalable system for detecting deepfake faces in videos. The proposed BEiT-HPR model incorporates a PatchReducer layer, which reduces the number of patches processed at different stages of the encoder. This approach lowers computational complexity by 62.2% while maintaining high detection accuracy. The model achieved impressive results in in-data evaluations, with accuracies of 83.92% on FF++, 97.59% on Celeb-DF, and 98.25% on DFD. It also performed well in cross-data evaluations, demonstrating its ability to handle data from different sources.

Despite these achievements, the research identified key limitations. The PatchReducer layer was only tested within the BEiT architecture. Its effectiveness on other transformer-based models has not been evaluated. Moreover, similar to the other deep learning models, the proposed method is limited in its ability to generalize to unseen datasets during cross-data evaluation. The model is trained on specific types of manipulations within the dataset, which allows it to perform well on similar data. However, when tested on new or unknown data, its performance decreases. This is a common challenge in detection systems as the unseen dataset can differ significantly from those in the training data due to variations in video compression, lighting conditions, and manipulation techniques. This limitation highlights the need for models that can adapt to a wider range of deepfake datasets without retraining. Another limitation is that the system is designed for binary classification, identifying whether a face is real or fake, but it cannot detect specific types of manipulations.

Additionally, the model focuses only on faces and does not analyze other manipulated elements in a video, such as backgrounds or objects.

Future work should focus on addressing these limitations. We aim to further optimize the PatchReducer layers to enhance efficiency without compromising detection accuracy. Potential improvements include incorporating adaptive patch reduction, multi-scale processing, and learnable patch selection. These techniques could enable dynamic adjustment of patch reduction based on input complexity, ensuring better feature retention while reducing computational costs. In addition, exploring the integration of PatchReducer into other transformer-based architectures could provide further insights into its generalizability and effectiveness in deepfake detection.

While this work focuses on improving deepfake detection efficiency, ethical considerations are also important. Ensuring privacy, fairness, and responsible use is crucial as these models are used in real-world applications. This model can be deployed in various environments where data privacy and bias matter. Future developments should follow ethical guidelines to prevent misuse, ensure fairness, and protect individual rights, especially in media, law enforcement, and public security.

Author Contributions Conceptualization, A.A.; methodology, A.A., M.A.B. and S.F.; software, A.A.; validation, A.A., M.A.B. and S.F.; data curation, A.A.; writing—original draft preparation, A.A.; writing—review and editing, A.A., M.A.B. and S.F.; visualization, A.A.

Declarations

Conflict of interest The authors have no Conflict of interest to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Abdullah M.T, Ali N.H.M (2023) Deepfake detection improvement for images based on a proposed method for local binary pattern of the multiple-channel color space. *International Journal of Intelligent Engineering & Systems* **16**(3)
- Abdullah M.T, Ali N.H.M (2024) Facial deepfake performance evaluation based on three detection tools: Mtcnn, dlib, and mediapipe. In: *AIP Conference Proceedings*, vol. 3097. AIP Publishing
- Arshed MA, Alwadain A, Faizan Ali R, Mumtaz S, Ibrahim M, Muneer A (2023) Unmasking deception: empowering deepfake detection with vision transformer network. *Mathematics* **11**(17):3710
- Amerini I, Galteri L, Caldelli R, Del Bimbo A (2019) Deepfake video detection through optical flow based cnn. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 0–0
- AIGov (2021) DeepFake Guide. <https://ai.gov.ae/wp-content/uploads/2021/07/AI-DeepFake-Guide-EN-2021.pdf>. [Accessed 24-11-2024]

- Almars AM (2021) Deepfakes detection techniques using deep learning: a survey. *J Comp Commun* 9(05):20–35
- Afchar D, Nozick V, Yamagishi J, Echizen I (2018) Mesonet: a compact facial video forgery detection network. In: 2018 IEEE International Workshop on Information Forensics and Security (WIFS), pp. 1–7. IEEE
- Bao H, Dong L, Piao S, Wei F (2021) Beit: Bert pre-training of image transformers. *arXiv preprint arXiv:2106.08254*
- Bayram B, Kunduracioglu I, Ince S, Pacal I (2025) A systematic review of deep learning in mri-based cerebral vascular occlusion-based brain diseases. *Neuroscience*
- Chollet F (2017) Xception: Deep learning with depthwise separable convolutions. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1251–1258
- Chougule A, Kolte S, Chamola V, Hussain A (2024) A novel generative adversarial network-based super-resolution approach for face recognition. *Exp Syst* 41(8):13564
- Chen H, Lin Y, Li B, Tan S (2022) Learning features of intra-consistency and inter-diversity: keys toward generalizable deepfake detection. *IEEE Trans Circuits Syst Video Tech* 33(3):1468–1480
- Cong P, Yang C (2023) Number of flops of training dnns for learning precoding. In: 2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring), pp. 1–6. IEEE
- Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S., et al (2020) An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*
- Demir I, Ciftci U.A (2021) Where do deep fakes look? synthetic face detection via gaze tracking. In: *ACM Symposium on Eye Tracking Research and Applications*, pp. 1–11
- Dufour N, Gully A (2019) Contributing Data to Deepfake Detection Research — research.google. <https://research.google/blog/contributing-data-to-deepfake-detection-research/>. [Accessed 07-09-2024]
- Das S, Jain T, Reilly D, Balaji P, Karmakar S, Marjit S, Li X, Das A, Ryoo M.S (2024) Limited data, unlimited potential: A study on vits augmented by masked autoencoders. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 6878–6888
- Dalal N, Triggs B (2005) Histograms of oriented gradients for human detection. In: 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), vol. 1, pp. 886–893. Ieee
- Dagar D, Vishwakarma DK (2024) Tex-net: texture-based parallel branch cross-attention generalized robust deepfake detector. *Multimed Syst* 30(4):233
- Deshmukh A, Wankhade SB (2020) Deepfake detection approaches using deep learning: a systematic review. *Intell Compt Netw: Proc IC-ICN 2020*:293–302
- Essa E (2024) Feature fusion vision transformers using mlp-mixer for enhanced deepfake detection. *Neuro-computing* 598:128128
- Fang X, Hao Y, Luo Y, Xu N, Cao J (2023) Forged facial video detection framework based on multi-region temporal relationship feature. *Aip Advances* 13(8)
- Gerritsen S (2022) A comparison of color space representations for the detection of deepfake images using deep learning. PhD thesis, Master's thesis, Tilburg University
- Ghita B, Kuzminykh I, Usama A, Bakhshi T, Marchang J (2024) Deepfake image detection using vision transformer models. In: *IEEE International Black Sea Conference on Communications and Networking*
- Guan L, Liu F, Zhang R, Liu J, Tang Y (2023) Mew: a generalizable deepfake detection method for few-shot learning. *Sensors* 23(21):8763
- Huang G, Liu Z, Van Der Maaten L, Weinberger K.Q (2017) Densely connected convolutional networks. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4700–4708
- Heidari A, Navimipour N.J, Dag H, Talebi S, Unal M (2024) A novel blockchain-based deepfake detection method using federated and deep learning models. *Cognitive Computation*, 1–19
- Ho H.-T, Nguyen L.V, Le T.H.T, Lee O.-J (2024) Face detection using eigenfaces: A comprehensive review. *IEEE Access*
- Han K, Xiao A, Wu E, Guo J, Xu C, Wang Y (2021) Transformer in transformer. *Adv Neural Inf Proc Syst* 34:15908–15919
- Kingra S, Aggarwal N, Kaur N (2023) Siamnet: exploiting source camera noise discrepancies using siamese network for deepfake detection. *Inf Sci* 645:119341
- Khan S.A, Dang-Nguyen D.-T (2023) Deepfake detection: Analysing model generalisation across architectures, datasets and pre-training paradigms. *IEEE Access*
- Kumar A, Kaur A, Kumar M (2019) Face detection techniques: a review. *Artif Intell Rev* 52:927–948
- Koopman M, Rodriguez A.M, Geradts Z (2018) Detection of deepfake video manipulation. In: *The 20th Irish Machine Vision and Image Processing Conference (IMVIP)*, pp. 133–136
- Kavthekar S, Vaidya S, Pujari V, Mane S (2023) Mim-vit: Deepfake detection using masked image modeling and vision transformer. In: *International Conference on Soft Computing for Problem-solving*, pp. 1–14. Springer

- Khormali A, Yuan J-S (2022) Dfdt: an end-to-end deepfake detection framework using vision transformer. *Appl Sci* 12(6):2953
- Khormali A, Yuan J-S (2024) Self-supervised graph transformer for deepfake detection. *IEEE Access*
- Khalil SS, Youssef SM, Saleh SN (2021) icaps-dfake: An integrated capsule-based model for deepfake image and video detection. *Future Int* 13(4):93
- Lei Ba J, Kiros JR, Hinton GE (2016) Layer normalization. *ArXiv e-prints*, 1607
- Lu T, Bao Y, Li L (2023) Deepfake video detection based on improved capsnet and temporal-spatial features. *CMC-Comp Mater Continua* 75(1):715–740
- Li L, Bao J, Yang H, Chen D, Wen F (2019) Faceshifter: Towards high fidelity and occlusion aware face swapping. *arXiv preprint arXiv:1912.13457*
- Li Y, Chang M.-C, Lyu S (2018) In icu oculi: Exposing ai generated fake face videos by detecting eye blinking. *arXiv preprint arXiv:1806.02877*
- Lin H, Huang W, Luo W, Lu W (2023) Deepfake detection with multi-scale convolution and vision transformer. *Dig Sig Proc* 134:103895
- Li A.S, Iyengar A, Kundu A, Bertino E (2024) Transfer learning for security: Challenges and future directions. *arXiv preprint arXiv:2403.00935*
- Lee G, Kim M (2021) Deepfake detection using the rate of change between frames based on computer vision. *Sensors* 21(21):7367
- Li Y, Lyu S (2018) Exposing deepfake videos by detecting face warping artifacts. *arXiv preprint arXiv:1811.00656*
- Liu Z, Lin Y, Cao Y, Hu H, Wei Y, Zhang Z, Lin S, Guo B (2021) Swin transformer: Hierarchical vision transformer using shifted windows. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10012–10022
- Lu W, Liu L, Zhang B, Luo J, Zhao X, Zhou Y, Huang J (2023) Detection of deepfake videos using long-distance attention. *IEEE Transactions on Neural Networks and Learning Systems*
- Li Y, Yang X, Sun P, Qi H, Lyu S (2020) Celeb-df: A large-scale challenging dataset for deepfake forensics. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3207–3216
- Mubarak R, Alsabou T, Alshaikh O, Inuwa-Dute I, Khan S, Parkinson S (2023) A survey on the detection and impacts of deepfakes in visual, audio, and textual formats. *IEEE Access*
- Mirzayev A, DİBEKLİOĞLU H (2024) Motion magnification-inspired feature manipulation for deepfake detection. *Turkish Journal of Electrical Engineering and Computer Sciences* 32(1), 166–182
- Maurício J, Domingues I, Bernardino J (2023) Comparing vision transformers and convolutional neural networks for image classification: a literature review. *Appl Sci* 13(9):5521
- Malik A, Kuribayashi M, Abdullahi SM, Khan AN (2022) Deepfake detection for human face images and videos: a survey. *Ieee Access* 10:18757–18775
- Mishra S.R, Mohapatra H, Edalatpanah S.A, Gourisaria M.K (2024) Advanced deepfake detection leveraging swin transformer technology. *Engineering Review: Međunarodni časopis namijenjen publiciranju originalnih istraživanja s aspekta analize konstrukcija, materijala i novih tehnologija u području strojarstva, brodogradnje, temeljnih tehničkih znanosti, elektrotehnike, računarstva i građevinarstva* 44(4-SI 2024), 45–56
- Mathews S, Trivedi S, House A, Povolny S, Fralick C (2023) An explainable deepfake detection framework on a novel unconstrained dataset. *Comp Intell Syst* 9(4):4425–4437
- Nguyen D, Astrid M, Ghorbel E, Aouada D (2024) Fakeformer: Efficient vulnerability-driven transformers for generalisable deepfake detection. *arXiv preprint arXiv:2410.21964*
- Ozdemir B, Pacal I (2025) An innovative deep learning framework for skin cancer detection employing convnextv2 and focal self-attention mechanisms. *Results in Eng* 25:103692
- Omar K, Sakr RH, Alrahmawy MF (2024) An ensemble of cnns with self-attention mechanism for deepfake video detection. *Neural Comp Appl* 36(6):2749–2765
- Pacal I (2024) Enhancing crop productivity and sustainability through disease identification in maize leaves: Exploiting a large dataset with an advanced vision transformer model. *Exp Syst Appl* 238:122099
- Patel M (2023) The Complete Guide to Image Preprocessing Techniques in Python — maahip1304. <https://medium.com/@maahip1304/the-complete-guide-to-image-preprocessing-techniques-in-python-dca30804550c>. [Accessed 12-09-2024]
- Pacal I, Kunderacioglu I, Alma MH, Deveci M, Kadry S, Nedoma J, Slany V, Martinek R (2024) A systematic review of deep learning techniques for plant diseases. *Artif Intell Rev* 57(11):304
- Pacal I, Ozdemir B, Zeynalov J, Gasimov H, Pacal N (2025) A novel cnn-vit-based deep learning model for early skin cancer diagnosis. *Biomed Sig Proc Control* 104:107627
- Patel Y, Tanwar S, Bhattacharya P, Gupta R, Alsuwian T, Davidson IE, Mazibuko TF (2023) An improved dense cnn architecture for deepfake image detection. *IEEE Access* 11:22081–22095

- Qin W, Lu T, Zhang L, Peng S, Wan D (2023) Multi-branch deepfake detection algorithm based on fine-grained features. *Computers, Materials & Continua* 77(1)
- Qin H, Zhou D, Xu T, Bian Z, Li J (2023) Factorization vision transformer: Modeling long-range dependency with local window cost. *IEEE Transactions on Neural Networks and Learning Systems*
- Rossler A, Cozzolino D, Verdoliva L, Riess C, Thies J, Nießner M (2019) Faceforensics++: Learning to detect manipulated facial images. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1–11
- Raza MA, Malik KM, Haq IU (2023) Holisticdfid: Infusing spatiotemporal transformer embeddings for deepfake detection. *Inf Sci* 645:119352
- Ramadhani K.N, Munir R, Utama N.P (2024) Improving video vision transformer for deepfake video detection using facial landmark, depthwise separable convolution and self attention. *IEEE Access*
- Rana MS, Nobl MN, Murali B, Sung AH (2022) Deepfake detection: a systematic literature review. *IEEE Access* 10:25494–25513
- Renggli C, Pinto A.S, Houlsby N, Mustafa B, Puigcerver J, Riquelme C (2022) Learning to merge tokens in vision transformers. *arXiv preprint arXiv:2202.12015*
- Sahla Habeeba M, Lijiya A, Chacko A.M (2021) Detection of deepfakes using visual artifacts and neural network classifier. In: *Innovations in Electrical and Electronic Engineering: Proceedings of ICEEE 2020*, pp. 411–422. Springer
- Suratkar S, Kazi F, Sakhalkar M, Abhyankar N, Kshirsagar M (2020) Exposing deepfakes using convolutional neural networks and transfer learning approaches. In: *2020 IEEE 17th India Council International Conference (INDICON)*, pp. 1–8. IEEE
- Saravana Ram R, Vinoth Kumar M, Al-shami T.M, Masud M, Aljuaid H, Abouhawwash M (2023) Deep fake detection using computer vision-based deep neural network with pairwise learning. *Intelligent Automation & Soft Computing* 35(2)
- Touvron H, Cord M, Sablayrolles A, Synnaeve G, Jégou H (2021) Going deeper with image transformers. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 32–42
- Thing V.L (2023) Deepfake detection with deep learning: Convolutional neural networks versus transformers. In: *2023 IEEE International Conference on Cyber Security and Resilience (CSR)*, pp. 246–253. IEEE
- The IoT Academy X (2024) Deepfake Detection Technology - Working| Uses| Tools — theiotacademy.co. <https://www.theiotacademy.co/blog/deepfake-detection/>. [Accessed 22-02-2025]
- Tolosana R, Romero-Tapiador S, Vera-Rodriguez R, Gonzalez-Sosa E, Fierrez J (2022) Deepfakes detection across generations: analysis of facial regions, fusion, and performance evaluation. *Eng Appl Artif Intell* 110:104673
- Usmani S, Kumar S, Sadhya D (2024) Efficient deepfake detection using shallow vision transformer. *Multimed Tools Appl* 83(4):12339–12362
- Ulmer A, Tong A (2023) Deepfaking it: America's 2024 election collides with AI boom. <https://www.reuters.com/world/us/deepfaking-it-americas-2024-election-collides-with-ai-boom-2023-05-30/>. [Accessed 24-11-2024]
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A.N, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Advances in neural information processing systems* 30
- Venkatachalam K, Trojovský P, Hubálovský Š (2023) Viola jones algorithm with capsule graph network for deepfake detection. *PeerJ Comp Sci* 9:1313
- Wang T, Cheng H, Chow KP, Nie L (2023) Deep convolutional pooling transformer for deepfake detection. *ACM Trans Multimed Compt, Commun Appl* 19(6):1–20
- Wang Z, Cheng Z, Xiong J, Xu X, Li T, Veeravalli B, Yang X (2024) A timely survey on vision transformer for deepfake detection. *arXiv preprint arXiv:2405.08463*
- Wang B, Huang L, Huang T, Ye F (2023) Two-stream xception structure based on feature fusion for deepfake detection. *Int J Comput Intell Syst* 16(1):134
- Xu Y, Li C, Li D, Sheng X, Jiang F, Tian L, Sirasao A (2023) Fdvit: Improve the hierarchical architecture of vision transformer. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5950–5960
- Zhang Z, Liu Q (2020) Detect video forgery by performing transfer learning on deep neural network. In: *Advances in Natural Computation, Fuzzy Systems and Knowledge Discovery: Volume 2*, pp. 415–422. Springer
- Zhao C, Wang C, Hu G, Chen H, Liu C, Tang J (2023) Istvt: interpretable spatial-temporal video transformer for deepfake detection. *IEEE Trans Inf Forensics Security* 18:1335–1348
- Zhao H, Zhou W, Chen D, Wei T, Zhang W, Yu N (2021) Multi-attentional deepfake detection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2185–2194
- Zhu Y, Zhang C, Gao J, Sun X, Rui Z, Zhou X (2024) High-compressed deepfake video detection with contrastive spatiotemporal distillation. *Neurocomputing* 565:126872

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Aneesa Al Redhaei¹ · Salam Fraihat¹ · Mohammed Azmi Al-Betar^{1,2}

✉ Mohammed Azmi Al-Betar
m.albetar@ajman.ac.ae

Aneesa Al Redhaei
202211654@ajmanuni.ac.ae

Salam Fraihat
s.fraihat@ajman.ac.ae

¹ Artificial Intelligence Research Center (AIRC), College of Engineering and Information Technology, Ajman University, Ajman, UAE

² Hourani Center for Applied Scientific Research, Al-Ahliyya Amman University, Amman, Jordan